

자기소개

원문 (236자)

가계부 포트폴리오 ○ ○ ○

Contact

- Phone : 010-0000-0000

- Email : abc@naver.com

[포트폴리오 목차]

클릭하시면 바로 넘어갑니다.

1. 프로젝트 개요

2. 프로젝트 설명

- 1. 사용 기술
- 2. 프로젝트 구조
- 3. ERD
- 4. 프로젝트 전체 구현 기능

3. 개인 내용

- 1. 직접 구현한 기능
- 2. 프로젝트에서 발휘한 역량과 성장한 점

[개선안 \(586자\)](#)

가계부 프로젝트 포트폴리오

지원자: ○ ○ ○

[Contact]

- Phone : 010-0000-0000

- Email : abc@naver.com

[포트폴리오 구성 안내]

본 포트폴리오는 가계부 개발 프로젝트의 전반적인 과정과 개인의 기여도를 체계적으로 정리한 문서입니다. 각 항목은 프로젝트의 개요부터 개인의 기술적 성장까지 흐름에 따라 서술형으로 구성되었습니다.

1. 프로젝트 개요 및 설명

가계부 프로젝트의 기획 의도와 목적을 정의하는 개요를 시작으로, 개발에 도입된 사용 기술을 명시합니다. 또한, 전체 시스템의 청사진을 보여주는 프로젝트 구조와 데이터베이스 설계를 확인할 수 있는 ERD를 포함하고 있습니다. 이를 바탕으로 애플리케이션에서 제공하는 프로젝트 전체 구현 기능의 범위를 구체적으로 설명합니다.

2. 개인 기여도 및 성장

프로젝트 내에서 본인이 주도적으로 담당하고 직접 구현한 기능에 대해 상세히 서술합니다. 단순한 기능 나열을 넘어, 해당 기능을 구현하는 과정에서 어떠한 기술적 판단과 사고 과정을 거쳤는지 정리하였습니다. 최종적으로 본 프로젝트를 통해 발휘한 역량과, 이를 통해 개발자로서 성장한 점을 회고합니다.

주요 변경사항

- 단순 나열된 목차를 '프로젝트 개요 및 설명', '개인 기여도 및 성장'이라는 그룹으로 묶어 서술형 문장으로 재구성
- 원문의 인적사항(Phone, Email) 및 수치(전화번호)는 정보 손실 없이 그대로 유지
- 포트폴리오 평가 기준에 맞춰 '직접 구현한 기능'과 '발휘한 역량'이 돋보일 수 있도록 각 목차가 가지는 목적을 구체화하여 서술

전후 점수 24.5 → 34.5 (+10.0) 미흡 → 미흡

A.과정/판단력 30→40(+10) | B.역할/기여도 20→30(+10) | C.성과/인사이트 10→20(+10) | D.작성품질 4→6(+2) | E.직무연관성 30→30(+0)

가계부 포트폴리오 (In-And-Out)

[개요_0]

원문 (2896자)

[프로젝트 개요]

| 항목 | 내용 |

| --- | --- |

| 프로젝트 소개 | 가계부와 일기를 함께 작성하는 웹어플리케이션 |

| 개발 인원 | 총 5명 (프론트엔드 2명 + 백엔드 3명) |

| 담당 역할 | 팀장, API 개발(수입, 보고서, 달력, 엑셀 출력), CI/CD 구성 및 RDS DB 세팅 |

| 개발 기간 | 총 42일 (2022-00-00 ~ 2022-00-00) |

| 성과 및 결과 | 기한 내 목표한 기능 구현 완료, CI/CD 및 실제 서버 배포 완료. |

- 프로젝트 깃헙 바로가기

- 배포된 웹 어플리케이션 바로가기

→ 테스트용 ID / PW : abc@kakao.com / password

[프로젝트 설명]

1) 사용 기술 가계부 포트폴리오 ○○○

1

2) 프로젝트 구조

3) ERD + 와이어 프레임

· ERD

#

와이어

프레임

<https://www.figma.com/file/kackFvkiQyAhxJnPNpjmJE/%EA%B0%80%EA%B3%84%EB%B6%80?node-id=2%3A4>

4) 프로젝트 전체 구현 기능 가계부 포트폴리오 ○ ○ ○

2

#

1.

회원

가입

및

로그인,

비밀

번호

찾기

<http://s3-west-2.amazonaws.com/secure.notion-static.com/fc5dd02a-17cd-443e-938c-47921cfcd129/%ED%A%8C%EC%9B%90.mp4>

▼ 회원 가입: 이메일을 인증하여 회원 가입한다.

- · 유효한 이메일 주소가 있으면 회원 가입이 가능하다.

- · 이메일 전송은 비동기 처리한다. 따라서, 회원 가입 제출하면 완료 창이 바로 뜬다.

- · 만약 회원 가입 버튼을 누르고 이메일이 전송됐는데 이메일이 삭제된 경우는?

- · 다시 회원 가입을 요청하거나 로그인을 시도하면 이메일을 재발송하여 계정을 활성화할 수 있도록 한다.

▼ 로그인: 일반 로그인과 SNS 로그인

- · 일반 로그인: In&Out 회원 가입을 통한 로그인
- · SNS 로그인: In&Out에 회원 가입을 하지 않아도 OAuth 2.0을 통해 구글, 네이버 계정으로 간편하게 로그인할 수 있다.

▼ 비밀번호 찾기(초기화): 등록된 이메일과 연락처를 통해 비밀번호를 초기화할 수 있다.

- · 이메일과 연락처를 확인해서 회원의 이메일로 비밀번호 초기화 링크가 전송된다.
- · 이메일 전송 비동기 처리: 회원 가입과 마찬가지로 비동기 처리하여 알림창이 바로 뜨도록 한다.

2. 수입, 지출, 달력(다이어리)

https://s3-us-west-2.amazonaws.com/secure.notion-static.com/82e9df3e-2e3a-4009-8d4a-23ff262723bd/%EB%85%B9%ED%99%94202211_17_23_49_14_762.mp4

수입 : 해당 날짜의 수입 사항을 적을 수 있다

- · 작성 후 저장 버튼을 누르면 서버에 해당 기록 저장됨
- · 선택 버튼을 누르고 해당 열 삭제 가능
- · 한 번에 여러 열 삭제 가능
- · 월 이동해서 그 달의 기록들을 확인 / 수정 1 삭제 가능

▼ 지출 : 해당 날짜의 지출 사항을 적을 수 있다

· 대부분의 사항 수입과 같음

▼ 달력: 해당 달의 수입/지출 기록 여부와 다이어리 등록 여부를 확인할 수 있다

- · 금일 시점의 달로 렌더링 됨
- · 상단에 그 달의 수입(in)과 지출(out)의 합계가 표시된다
- · 금일 날짜에 색이 들어가서 확인 가능
- · 수입/지출 가계부가 기록된 날짜에는 돋보기 아이콘이 표시되며 클릭 시 수입/지출 표가 출력 된다 (마우스 위치에 따라 수입/지출 표의 출력 위치가 바뀌며 X자를 누르거나 표 바깥 부분을 누르면 창이 사라진다)
- · 각 날짜를 누르면 각 날짜 별 다이어리 창이 화면에 나타나고 그 날의 일기를 등록 / 수정 / 삭제 할 수 있다
- · 일기가 등록된 날짜에는 다이어리 아이콘이 표시되며 사진의 유무에 따라 주황색 / 회색으로 구분 가능
- · 상단의 화살표로 전 달과 다음 달로 이동 가능
- · 화면 상에 나타나지만 해당 달에 속하지 않은 날짜로는 클릭으로 접근이 불가능하다

▼ 다이어리: 해당 날짜로 기록해둔 수입/지출 확인 가능하며 짧게 메모할 수 있다

- · 왼쪽 표로 해당 날짜의 수입/지출 표를 볼 수 있다 가계부 포트폴리오 ○○○

3

- · 왼쪽 표의 여백 공간은 수입/지출의 요소의 수에 따라 계산되어 출력 된다
- · 오른쪽 공간에서 다이어리를 작성할 수 있다
- · 이미지를 등록할 수 있으며 수정으로 들어가 이미지 상단에 X자를 누르면 등록하려고 했던 이미지가 삭제된다
- · 오른쪽 공간 상단의 화살표 아이콘을 눌러 이전 날과 다음 날의 일기 페이지로 이동 가능

- 오른쪽 공간 상단의 날짜를 눌러 달력이 뜨면 원하는 날짜의 일기 페이지로 이동 가능
- 최상단의 X자를 누르거나 다이어리의 바깥 부분을 클릭하면 다이어리 창이 사라진다

3. 월간 / 연간보고서
<https://s3-us-west-2.amazonaws.com/secureotion-static.com/af435-b533-1fceb63cccb/%E1%84%87%E1%85%A9%E1%84%80%E1%85%A9%E1%84%89%E1%85%A5%E1%84%8B%E1%85%A7%E1%86%B>
 C%E1%84%89%E1%85%A1%E1%86%BC1080.mp4

▼ 월간 보고서 : 현재 월에 해당하는 수입/지출 데이터를 카테고리 별로 분류한 차트를 볼 수 있다.

- 월 단위로 날짜를 이동할 수 있다.
- 차트를 막대형, 파이형으로 볼 수 있고 카테고리 별로 금액과 비율을 알 수 있다.
- 월간 보고서를 엑셀파일로 다운로드할 수 있다.

▼ 연간 보고서 : 현재 월 기준 12개월 전의 수입/지출 데이터를 매월 카테고리 별로 분류한 그래프, 표를 볼 수 있다.

개선안 (1012자)

[프로젝트경험 — 가계부·다이어리 통합 웹 애플리케이션]

팀장 및 백엔드 개발자로서 총 5명(프론트엔드 2명, 백엔드 3명)의 팀원과 함께 42일간 수입/지출 내역과 일기를 통합 관리하는 웹 애플리케이션을 구축했습니다. 제한된 기간 내에 안정적인 배포 환경을 마련하고, 복잡한 데이터 집계 요구사항을 해결하는 것이 핵심 과제였습니다.

가장 집중했던 부분은 월간 보고서 및 최근 12개월 기준의 연간 보고서 생성 시 발생하는 복잡한 데이터 집계 로직의 설계였습니다. 저는 유지보수성과 확장성을 고려해 서비스 계층을 세분화하는 다계층 아키텍처를 도입했습니다. 'Report Controller'가 'Report Service(Layer 1)'를 호출하고, 이는 다시 'Income/Expense Service(Layer 2)'를 거쳐 데이터를 조회하도록 흐름을 제어했습니다. 특히 데이터베이스 접근(Query Repository) 권한을 Service Layer 2로만 제한하여 계층 간 책임을 명확히 하고 결합도를 낮췄습니다. 또한, Hibernate ORM을 활용해 카테고리별 수입 및 지출 합계 산출 쿼리를 최적화했습니다.

API 기능 구현에 있어서는 수입, 달력, 보고서 및 엑셀 출력 로직을 전담했습니다. 특히 엑셀 다운로드 기능은 ColConfig 객체를 활용해 컬럼 헤더와 데이터 매핑 로직을 컴포넌트화하여 코드의 재사용성을 높였습니다. 더불어 42일의 짧은 개발 기간 내 협업 효율을 극대화하기 위해 인프라를 직접 세팅했습니다. AWS EC2와 RDS(MySQL)를 기반으로 데이터베이스 환경을 구축하고, GitHub, Jenkins, Docker를 연동한 CI/CD 파이프라인을 완성해 개발부터 실제 서버 배포까지의 과정을 자동화했습니다.

결과적으로 기한 내에 목표한 모든 API 개발과 서버 배포를 완수했습니다. 이 프로젝트를 통해 명확한 계층형 아키텍처 설계가 복잡한 비즈니스 로직 처리에 주는 안정감과, 자동화된 파이프라인이 팀 생산성에 미치는 긍정적인 영향을 깊이 체득했습니다.

주요 변경사항

- 단순 기능 매뉴얼식 나열을 삭제하고 백엔드 아키텍처와 인프라 중심의 서술로 전면 수정
- 이미지 컨텍스트를 반영하여 Service Layer 1/2 분리 및 Query Repository 접근 제한 설계의 이유와 목적 명시
- 엑셀 출력 컴포넌트화(ColConfig) 및 Hibernate 쿼리 활용 과정을 구체적으로 추가
- 42일, 5명(프론트 2명/백엔드 3명), 12개월 등 원문의 정량적 수치를 유지하며 성과와 연결
- Jenkins, Docker, AWS EC2/RDS 등 CI/CD 환경 구축 과정과 효과를 팀장 역할과 연계하여 서술

전후 점수 46.5 → 85.8 (+39.2) 미흡 → 우수 ★★★

[개요_1]

원문 (828자)

- 월 단위로 날짜를 이동할 수 있다.
- 그래프는 꺾은선 그래프로 카테고리 선택할 수 있다.
- 그래프는 월마다 카테고리에 대한 값의 합산, 합산에 대한 증감을 알 수 있다.
- 표는 월마다 수입/지출 모든 카테고리에 대한 값의 합산을 볼 수 있다.
- 연간 보고서를 엑셀파일로 다운로드할 수 있다.

4. 회원 수정, 비밀번호 변경, 회원 탈퇴
https://s3-us-west-2.amazonaws.com/secure.notion-static.com/0299ca56-6dbb-44f9-90e8-a37878dd8ebe/%E1%84%92%E1%85%AC%E1%84%8B%E1%85%AF%E1%86%AB%E1%84%89%E1%85%A6%E1%84%90%E1%85%B5%E1%86%BC1080.mp4

▼ 회원 수정 : 이메일을 제외한 프로필 사진 및 정보를 변경할 수 있다.

- 일반 로그인 사용자는 회원가입 시 입력한 정보를 볼 수 있고 수정할 수 있다.
- OAuth 로그인 사용자는 이메일, 닉네임, 전화번호를 볼 수 있고 수정할 수 있다.
- 프로필 사진을 등록, 수정, 삭제할 수 있다.

▼ 비밀번호 변경 : 현재 비밀번호와 새 비밀번호를 입력하여 변경할 수 있다.

- 일반 로그인 사용자는 비밀번호 변경을 이용할 수 있다.
- OAuth 로그인 사용자는 이용할 수 없다.

▼ 회원 탈퇴 : 회원 탈퇴할 수 있다.

- 일반 로그인 사용자는 비밀번호를 이용하여 탈퇴할 수 있다
- OAuth 로그인 사용자는 바로 탈퇴할 수 있다.

가계부 포트폴리오 ○○○

4

- 탈퇴하게 되면 다시 로그인 할 수 없다.

개선안 (1023자)

[프로젝트 경험 — 가계부 시스템 통계 보고서 및 회원 관리 기능 구현]

1. 다계층 아키텍처 기반의 보고서 및 통계 기능 구현

가계부 사용자가 월 단위로 재정 상태를 파악할 수 있도록 꺾은선 그래프(카테고리별 합산 및 증감)와 표(전체 수입/지출 카테고리 합산)를 제공하는 통계 기능을 구현했습니다. 복잡한 데이터 집계를 효율적으로 처리하기 위해 계층형 아키텍처를 도입했습니다. Report Controller가 Report Service를 호출하면, 하위 계층인 Income Service와 Expense Service를 통해 Query Repository에서 데이터를 분리하여 조회하도록 계층 간 역할을 명확히

설계했습니다. 또한, 연간 보고서를 엑셀 파일로 다운로드할 수 있도록 ColConfig 객체로 엑셀 컬럼 헤더와 너비를 세밀하게 정의하고, excelDownloadComponent 를 활용해 데이터를 처리했습니다. 이 과정에서 예외 발생 시 디버깅 로그를 출력하고 null을 반환하도록 하여 서버의 안정성을 확보했습니다.

2. 인증 방식에 따른 회원 관리 로직 분기 및 이슈 분석

일반 로그인과 OAuth 로그인 사용자의 특성을 구분하여 프로필 수정, 비밀번호 변경, 탈퇴 로직을 각각 맞춤형으로 구현했습니다. 일반 사용자는 가입 정보 전체 조회/수정, 비밀번호 변경, 그리고 비밀번호 검증 기반의 탈퇴가 가능하도록 구성했습니다. 반면 OAuth 사용자는 이메일, 닉네임, 전화번호 등 제한적인 정보만 수정할 수 있게 하고, 즉시 탈퇴가 이루어지도록 로직을 분리했습니다. 프로필 사진의 등록, 수정, 삭제 기능도 공통으로 적용했습니다. 개발 과정 중 비밀번호 변경을 위한 PATCH 요청에서 CORS 정책 위반으로 네트워크가 차단되는 이슈를 브라우저 콘솔을 통해 발견 및 분석하며, 클라이언트와 서버 간 보안 정책 통신의 중요성을 파악했습니다. 아울러 칸반 보드와 타임라인 캘린더를 통해 스프린트 계획과 데일리 스크럼 등 애자일 방식으로 일정을 체계적으로 추적하며 프로젝트를 수행했습니다.

주요 변경사항

- 단순 기능 나열을 기술적 구현 방식과 구조 중심으로 서술 개편
- Report Controller와 하위 Service(Income/Expense)를 거치는 다계층 아키텍처 데이터 조회 과정 추가
- ColConfig 및 excelDownloadComponent를 활용한 엑셀 다운로드 구현 및 예외 처리 과정 명시
- 일반 사용자 및 OAuth 로그인 방식에 따른 회원 관리/탈퇴 로직 분기 구체화
- 개발 중 발견한 CORS 정책 차단 오류 분석 경험 추가
- 칸반 보드와 타임라인을 활용한 애자일 기반 프로젝트 관리 경험 반영

전후 점수 30.5 → 76.8 (+46.2) 미흡 → 양호 ★★

A.과정/판단력 30→75(+45) | B.역할/기여도 20→80(+60) | C.성과/인사이트 10→60(+50) | D.작성품질 9→10(+1) | E.직무연관성 40→85(+45)

[개발]

원문 (2660자)

[개인 내용]

1) 직접 구현한 기능

1. 수입 기능 (Income API Statement)

```
| Aa URL | [c]▼[c] 메소드 | 상 태 | = body | response | error | 설 명 | creator | API TEST | 20 front |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| api/income? startDt=""&endDt="" | GET | Done | | status, List | 1. member_validation 2. start_dt, end_dt 입력 x |
수 입 리 스 트 가 저 오 기 | | Done | |
| api/income | POST | Done | List | status, message | 1. member validation 2. (메모 제외) 값이 없는 경우 | 수 입 저 장 및
수 정 | | Done | |
| api/income | DELETE | Done | List | status, message | 1. member validation 2. (메모 제외) 값이 없는 경우 | 수 입 삭 제
| | Done | |
```

2. 보고서 조회 중 월간수입, 연간조회 (Report API Statement)

```
| Aa URL | [c]▼[c] 메소드 | 상 태 | body | = response | error | = 설명 | creator | API TEST | 22 front |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| api/report/expense/month? startDt=""&endDt="" | GET | Done | | status, List | 1. member_validation 2. start_dt,
```

end_dt 입력 x | 월간 보고서 가져오기 | | Done | |
 | api/report/income/month? startDt=""&endDt="" | GET | Done | | status, List | 1. member validation 2. start_dt, end_dt 입력 X | 월간 수입 보고서 가져오기 | | Done | |
 | api/report/year? startDt=""&endDt="" | GET | Done | | status, List | 1. member validation 2. start_dt, end_dt 입력 X | 연간 보고서 가져오기 | | Done | |

3. 달력 조회 기능 (Calendar API Statement)

Aa URL	메소드	상태	body	response	error	설명	22 creator	API TEST	22 front
api/calendar? startDt=""&endDt=""	GET	Done		status, List	1. member validation 2. start_dt, end_dt 입력 X	달력에 날마다 수입, 지출 리스트 가져오기		Done	

4. 엑셀 출력 기능 (Excel API Statement)

Aa URL	메소드	상태	body	response	error	설명	20 creator	API TEST	22 front

가계부 포트폴리오 ○ ○ ○

5

- 프로젝트 전체 노선 페이지 기획 및 구성함
- 백엔드 서버 및 DataBase에 대한 AWS 배포 전반을 담당함
- 주 단위의 스프린트 계획 및 회고 주도함
- 칸반보드를 활용하여 프로젝트 전체 일정 관리하여 일정 내에 구현을 완성함
- 백엔드 API의 1/3을 담당함

Aa URL	[c]▼[c] 메소드	상태	body	response	error	설명	22 creator	API TEST	22 front
api/excel/income? startDt=""&endDt=""	GET	Done		excel download 창 팝업	1. member validation	startDt ~ endDt까지의 수입내역을 Excel로 Export		Done	
api/excel/expense? startDt=""&endDt=""	GET	Done		excel download 창 팝업	1. member validation	startDt ~ endDt까지의 지출내역을 Excel로 Export		Done	
api/excel/year	POST	Done	startDt, List	excel download 창 팝업	1. member validation	startDt부터 1년치의 연간보 고서를 Excel로 Export		Done	

개선안 (778자)

[프로젝트경험 — 가계부 및 재정 관리 플랫폼]

백엔드 개발 및 인프라 배포, 협업 프로세스 관리를 주도하며 전체 백엔드 API의 1/3을 직접 설계하고 구현했습니다.

▶ 다계층 아키텍처 기반 API 설계

수입 관리, 월간/연간 보고서, 달력 조회 및 엑셀 다운로드 API를 전담하여 개발했습니다. 복잡한 데이터를 집계하는 보고서 기능의 유지보수성을 높이기 위해 다계층(Layered) 아키텍처를 도입했습니다. 단일 서비스에서 모든 로직을 처리하는 대신, Report Service 산하에 Income Service와 Expense Service를 분리하고 각각의 Query Repository를 통해서만 데이터에 접근하도록 계층 간 책임과 접근 규칙을 명확히 정의했습니다. 또한, 기간별 내역 및 1년 치 연간 보고서를 추출하는 엑셀 Export 기능은 내부 객체로 컬럼 헤더를 동적으로 구성하고, 다운로드 중 발생할 수 있는 예외 처리 로직을 꼼꼼히 반영하여 안정성을 높였습니다.

▶ AWS 인프라 구축 및 애자일 기반 프로젝트 관리

백엔드 서버 및 데이터베이스(DB)에 대한 AWS 배포 전반을 단독으로 담당하여 안정적인 운영 환경을 구축했습니다. 더불어 팀의 개발 생산성을 높이기 위해 전체 Notion 페이지 구조를 기획하고, 주 단위의 스프린트 계획과 회고를 직접 주도했습니다. 칸반보드와 타임라인을 활용하여 PR 및 Merge 등 태스크 진행 상황을 꼼꼼히 시각화하여 관리한 결과, 개발 병목을 방지하고 계획된 일정 내에 모든 시스템 구현을 성공적으로 완수할 수 있었습니다.

주요 변경사항

- 표 형태의 API 목록을 아키텍처 설계 및 구현 과정이 드러나는 서술형으로 재구성
- 이미지 컨텍스트를 활용해 다계층(Layered) 아키텍처 도입 근거 및 데이터 접근 분리 과정 추가
- 단순히 나열되었던 프로젝트 관리 역할(노션, 칸반, 스프린트)을 팀 협업 최적화 및 일정 준수 성과와 연결
- 엑셀 다운로드 기능에 컬럼 헤더 구성 및 예외 처리 등 코드 기반의 구현 디테일 보완
- 원문에 명시된 주요 수치(백엔드 API의 1/3, 1년 치 연간 보고서)를 삭제 없이 문맥 내에 자연스럽게 배치

전후 점수 36.5 → 78.5 (+42.0) 미흡 → 양호 ★★

A.과정/판단력 30→75(+45) | B.역할/기여도 40→85(+45) | C.성과/인사이트 20→65(+45) | D.작성품질 7→10(+3) | E.직무연관성 50→80(+30)

[성과_0]

원문 (2985자)

2) 프로젝트에서 발휘한 역량과 성장한 점 아래 내용은 모두 제가 주도적으로 직접 경험하고 해결한 내용입니다.

프로젝트에서의 높은 기여도: 35 / 100 (총 5명, 주관적인 점수)

엑셀 출력기능에서의 컴포넌트 설계

- · 엑셀 출력기능에서의 컴포넌트 설계시 해당 기능을 1회성이 아닌 Component로 두어 확장성을 고려함

- · 의존성 분리를 위해 Component는 엑셀 출력을 위한 데이터를 인자로 받으면 엑셀을 출력해주는 로직만을 담도록 구성

- · Service 클래스에서는 첫 행의 컬럼정보만 세팅해주면 그에 맞는 엑셀과 데이터가 추출되도록 구현

- · 이 과정에서 필요한 apache poi 라이브러리를 새로 학습하여 사용하였으며, 구글링해서 알게 된 내용들을 가지고 스스로 컴포넌

- 트 코드를 구성함

| 조회기간: 2021-11-01 2022-10-31 | 조회기간: 2021-11-01 2022-10-31 | 조회기간: 2021-11-01 2022-10-31 |
조회기간: 2021-11-01 2022-10-31 | 조회기간: 2021-11-01 2022-10-31 | 조회기간: 2021-11-01 2022-10-31 |
조회기간: 2021-11-01 2022-10-31 | 조회기간: 2021-11-01 2022-10-31 | 조회기간: 2021-11-01 2022-10-31 |
조회기간: 2021-11-01 2022-10-31 | 조회기간: 2021-11-01 2022-10-31 | 조회기간: 2021-11-01 2022-10-31 |
조회기간: 2021-11-01 2022-10-31 | 조회기간: 2021-11-01 2022-10-31 |

| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |

| 항목 | 2021-11 | 2021-12 | 2022-01 | 2022-02 | 2022-03 | 2022-04 | 2022-05 | 2022-06 | 2022-07 | 2022-08 |
2022-09 | 2022-10 | 합계 |

| 수입지출합계 | 0 | 0 | 1,247,330 | 1,258,400 | 2,566,450 | 4,465,590 | 5,038,390 | 7,799,210 | 2,000,640 | 2,555,980 | 6,399,740 | -1,536,460 | 31,795,270 |

| 수입합계 | 0 | 0 | 2,939,330 | 2,828,400 | 2,872,450 | 5,348,590 | 5,526,890 | 8,148,710 | 3,018,640 | 2,869,980 | 6,830,740 | 9,673,440 | 50,057,170 |

| 추수업 | 수 | 0 | 2,938,830 | 2,828,400 | 2,800,000 | 5,298,590 | 5,285,190 | 4,014,800 | 2,818,640 | 2,801,400 | 6,830,740 | 9,656,200 | 45,272,790 |

| 부수입 | 0 | 0 | 500 | 0 | 72,450 | 50,000 | 241,700 | 3,876,910 | 0 | 18,500 | 0 | 0 | 4,260 | 140 |

| 전월이월 | 수 | 0 | 0 | 0 | 0 | 0 | 수 | 257,000 | 0 | 0 | 0 | 총 | 257,000 |

| 저축/보험 | 0 | 0 | 0 | 0 | 0 | 0 | 200,000 | 50,000 | 0 | 17,240 | 267,240 |

| 지출함께 | 0 | 0 | 1,692,000 | 1,570,000 | 306,000 | 883,000 | 488,500 | 549,500 | 1,018,000 | 314,000 | 431,000 | 11,209,900 | 18,261,900 |

| 주거/통신 | 수 | 0 | 640:000 | 615,000 | 40,000 | 570,000 | 70,000 | 20,000 | 550:000 | 60,000 | 25,000 | 622,000 | 3,212,000 |

| 건강/문화 | 0 | 0 | 343,000 | 273,000 | 33,000 | 10,000 | 220,000 | 10,000 | 280,000 | 30,000 | 290,000 | 9,245,000 | 10,734,000 |

| 경조사/외비 | 수 | 0 | 190,000 | 140,000 | 30,000 | 100,000 | 100,000 | 140,000 | 100,000 | 0 | 0 | 270,000 | 1,070,000 |

| 식비 | 0 | 0 | 179,000 | 127,000 | 11,000 | 33,000 | 17,500 | 7,500 | 17,000 | 7,000 | 31,000 | 57,000 | 487,900 |

| 생활용품 | 수 | 0 | 108,000 | 73,000 | 50,000 | 50,000 | 1,000 | 52,000 | 1,000 | 52,000 | 3,000 | 508,000 | 898,000 |

| 교통/차량 | 0 | 0 | 90,000 | 120,000 | 50,000 | 70,000 | 0 | 70,000 | 70,000 | 0 | 70,000 | 80,000 | 620,000 |

| | | | | | | | | | | | | 100,000 | |

| 교육/육아 외복/미용 | 수 | 0 | 0 | 70,000 | 80,000 | 80,000 | 0 | 50,000 | 80,000 | 0 | 50,000 | 0 | 0 | 80,000 | 70,000 | 0 | 12,000 | 317,000 | 490,000 | 70% | 000 |

| 세금/이자 | 0 | 0 | 0 | 52,000 | 20,000 | 142,000 | 0 | 12,000 | 0 | 0 | 0 | 0 | 15,000 | 0 | 10,000 | 45,000 |

연간 보고서를 엑셀로 출력한 화면 예시

ExcelService.java 코드

[ExcelService.java]

개선안 (779자)

[프로젝트경험 — 재정 관리 연간 보고서 엑셀 컴포넌트 설계]

총 5명이 참여한 프로젝트에서 35%의 기여도를 담당하며, 개인의 수입 및 지출 내역을 연간 보고서 형태의 엑셀로 다운로드하는 기능을 주도적으로 개발했습니다. 여러 데이터 도메인에서 엑셀 출력 요구사항이 발생할 수 있는 상황이었습니다. 이를 개별적인 1회성 로직으로 작성할 경우 코드 중복과 유지보수 비용이 증가할 것이라 판단하여, 재사용성과 확장성을 최우선으로 한 공통 Component 설계를 목표로 삼았습니다.

가장 집중한 부분은 Service 클래스와 엑셀 렌더링 로직 간의 의존성 분리였습니다. 저는 Apache POI 라이브러리를 학습하여 엑셀 생성만을 전담하는 독립적인 컴포넌트를 구성했습니다. Service 계층에서는 ColConfig 객체를 활용해 엑셀의 첫 행에 들어갈 컬럼 정보(이름, 너비 등)와 추출할 데이터만을 세팅하여 인자로 전달하도록 구조화했습니다. 이를 통해 컴포넌트는 전달받은 데이터의 형태에 구애받지 않고 유연하게 엑셀 파일을 출력할 수 있게 되었습니다.

이러한 구조적 분리를 통해 Service 계층은 비즈니스 로직에만 온전히 집중할 수 있게 되었습니다. 또한, 추후 다른 형식의 엑셀 출력 기능이 필요할 때 렌더링 코드를 중복해서 작성할 필요 없이, 컬럼 정보와 데이터 주입만으로 즉각적인 확장이 가능한 유연한 코드를 완성했습니다. 이 과정을 통해 단일 책임 원칙(SRP)을 실무 코드에 적용해 보고, 모듈의 응집도를 높이고 결합도를 낮추는 아키텍처 설계의 중요성을 깊이 체감했습니다.

주요 변경사항

- 단순 글머리 기호 형태의 나열을 문제 정의, 판단, 실행, 인사이트 흐름의 논리적 서술로 변경
- 이미지 컨텍스트에서 도출한 도메인(수입/지출 연간 보고서)과 기술적 상세(ColConfig 객체)를 추가해 내용 구체화

- 비전문적인 표현('구글링해서')을 제거하고, 의존성 분리와 공통 모듈화라는 백엔드 핵심 역량으로 연결
- 원문의 수치(총 5명, 기여도 35)를 프로젝트 배경 설명에 자연스럽게 배치하여 보존

전후 점수 57.2 → 82.8 (+25.5) 보통 ★ → 양호 ★★

A.과정/판단력 60→85(+25) | B.역할/기여도 55→80(+25) | C.성과/인사이트 40→70(+30) | D.작성품질 8→10(+2) | E.직무연관성 65→90(+25)

[성과_1]

원문 (500자)

// 컬럼의 이름과 셀의 넓이를 클래스에 담아서 컴포넌트에 넘길 수 있도록 구현함.

```
public static class ColConfig {
```

```
    public String colName ;
```

```
    public int colWidth;
```

```
}
```

```
public ResponseEntity<InputStreamResource> downloadIncomeExcelFile(
```

```
    HttpServletRequest request, String email, LocalDate startDt, LocalDate endDt)
```

```
    throws IOException {
```

```
    Member member = findMemberByEmail(email) ;
```

```
    // ExcelComponent에 파라미터로 넘길 데이터도 메서드를 통해서 만들어옴.
```

```
    List<IncomeExcelDto> excelRowData = setIncomeExcelDataList(email, startDt, endDt);
```

가계부 포트폴리오 ○ ○ ○

6

개선안 (788자)

[가계부 포트폴리오 — 소득 내역 엑셀 다운로드 기능 설계 및 구현]

가계부 서비스에서 사용자가 특정 기간의 소득 내역을 엑셀 파일로 추출할 수 있도록 Spring Boot 환경에서 다운로드 기능을 구현했습니다.

기능 구현 과정에서 단순히 데이터를 엑셀로 변환하는 데 그치지 않고, 코드의 유지보수성과 향후 기능 확장을 고려하여 재사용 가능한 컴포넌트 기반으로 구조를 개선했습니다. 구체적으로, 엑셀의 컬럼 이름(colName)과 셀의 너비(colWidth)와 같은 포맷 설정 정보를 ColConfig 라는 별도의 클래스에 담아 관리하도록 책임을 분리했습니다.

사용자의 이메일을 통해 회원(Member) 정보를 조회하고, 요청된 시작일(startDt)과 종료일(endDt)을 기준으로 데이터를 IncomeExcelDto 리스트로 가공했습니다. 이후, 별도로 구현한 엑셀 생성 메서드를 통해 준비된 데이터와 ColConfig 설정값을 공통 컴포넌트인 ExcelComponent 에 파라미터로 넘겨 처리를 위임했습니다. 최종적으로 생성된 파일은 클라이언트가 안정적으로 다운로드할 수 있도록 InputStreamResource 로 변환하여 ResponseEntity 객체로 반환했습니다.

이러한 구조적 분리를 통해 엑셀 파일 생성 로직과 비즈니스 데이터 가공 로직의 결합도를 낮출 수 있었습니다. 설정값과 데이터를 독립적으로 다룸으로써, 추후 지출 내역 등 다른 데이터에 대한 엑셀 다운로드 기능이 요구되더라도 기존 컴포넌트를 쉽게 재사용하고 유연하게 확장할 수 있는 기반을 마련했습니다.

주요 변경사항

- 단순 코드 나열과 주석을 포트폴리오에 적합한 서술형 문장으로 전면 재구성
- ColConfig 클래스와 ExcelComponent 사용 목적을 '재사용성과 유지보수성 확보'라는 판단 근거로 연결
- 데이터 조회(findMemberByEmail)와 DTO(IncomeExcelDto) 변환 과정을 개발 과정으로 구체화
- ResponseEntity와 InputStreamResource를 반환하는 코드를 클라이언트 응답 처리 맥락으로 풀어내어 추가

전후 점수 40.5 → 81.8 (+41.2) 미흡 → 양호 ★★

A.과정/판단력 40→85(+45) | B.역할/기여도 30→80(+50) | C.성과/인사이트 20→65(+45) | D.작성품질 10→10(+0) | E.직무연관성 50→90(+40)

[이슈]

원문 (1424자)

▼ 쿼리 및 성능 개선

· 문제상황 및 원인

。 처음 기능 구현 시 쿼리를 생각하지 않고 로직을 구현한 것이 원인. 아무리 정보가 많이 필요한 연간보고서라도 쿼리가 50개가 나갔음.

-- 한 달의 내역을 가져오는 메서드를 연간에서 재사용 → 쿼리 X12

-- 한 달의 내역을 가져올 때 내부적으로 합계를 구하는 것을 쿼리로 했음 → 쿼리 X2

-- 위 작업이 수입, 지출로 나뉘져 있었음 → 쿼리 X2

-- 멤버 Validation에서 쿼리 2개 가계부 포트폴리오 ○ ○ ○

7

· 해결방법 및 배운점

[월간 보고서 조회 개선작업]

-- 서비스 로직 변경 (합계 구하는 쿼리 대신 List loop 돌면서 합계 계산)

-- Report Controller, Service Layer 1, Service Layer 2의 기존 테스트코드 그대로 통과

[연간 보고서 조회 개선작업]

-- YearlyReportDto 구현 및 쿼리 메서드 구현

-- Service Layer 2 서비스 메서드 구현

-- 기존 Controller와 Report Service 테스트코드는 수정 없이 통과

- 。 의존성을 줄여서 코드를 작성하는 것의 효율성을 체감함.

- 가장 뿌듯했던 것은, 쿼리 메서드 수정 부분을 제외하고는 테스트코드의 수정 거의 없이 재통과 가능했고, 바로 프로젝트에 적용
- 용할 수 있었던 부분
- 나의 기존 코드와 로직에 간혀있음을 인지하였고,
- 과감하게 관점을 바꿀 용기를 내지 않으면 내 코드와 발전을 막을 수 있음을 깨달음.

· 성능 개선 가계부 포트폴리오 ○ ○ ○

8

	Monthly	Monthly	Monthly	Yearly	Yearly	Yearly
기존 변경 개선비율 기존 변경 개선비율	3	2	33%	50	4	92%
쿼리개수	3	2	33%	50	4	92%
평균(ms)	124.6	66.1	47%	2875.5	149.1	95%
호출1	262	159		4841	232	
호출2	112	84		2888	117	
호출3	119	33		1217	137	
호출4	116	47		899	133	
호출5	116	48		841	148	
호출6	118	58		732	133	
호출7	91	55		6159	144	
호출8	98	59		8451	144	
호출9	106	66		2054	136	
호출10	108	52		673	167	

개선안 (1027자)

[프로젝트경험 — 가계부 보고서 조회 성능 개선]

가계부 서비스의 연간 보고서 조회 시 최대 50개의 쿼리가 발생하며 응답 시간이 평균 2875.5ms까지 지연되는 성능 문제를 분석하고 개선했습니다. 초기 구현 시 월간 내역 조회 메서드를 연간 조회에 재활용하여 12번의 반복 호출(x12)이 발생했고, 수입과 지출을 분리한 조회(x2), 쿼리를 통한 합계 계산(x2), 그리고 중복된 멤버 검증 쿼리(2개)가 누적되어 발생한 문제였습니다.

이를 해결하기 위해 데이터베이스 접근 비용을 줄이고 애플리케이션 계층의 데이터 처리 역할을 강화했습니다. 첫째, 월간 보고서는 데이터베이스에서 합계를 구하는 별도의 쿼리를 제거하고, 데이터를 한 번에 조회한 후 서비스 로직에서 List를 순회(Loop)하며 합계를 직접 계산하도록 변경했습니다. 둘째, 연간 보고서는 YearlyReportDto 를 새롭게 도입하고, 하위 서비스 계층(Service Layer 2)과 Query Repository를 활용해 집계된 데이터를 한 번에 가져오는 전용 쿼리 메서드를 구현했습니다.

이러한 개선 결과, 연간 보고서 조회 쿼리 개수를 50개에서 4개로 92% 줄였고, 평균 응답 시간을 2875.5ms에서 149.1ms로 95% 단축하는 성과를 거두었습니다. 월간 보고서 역시 쿼리를 3개에서 2개로 33% 줄여 평균 응답 시간을 124.6ms에서 66.1ms로 47% 개선했습니다.

특히 Controller - Service Layer 1 - Service Layer 2 - Repository로 이어지는 다계층 아키텍처의 의존성을 명확히 분리한 덕분에, 데이터 조회 로직을 대폭 수정했음에도 상위 계층(Controller, Service Layer 1)의 기존 테스트 코드를 수정 없이 모두 통과하며 안정적으로 적용할 수 있었습니다. 이 경험을 통해 모듈 간 결합도를

낮추는 설계가 리팩토링 시 얼마나 큰 효율을 제공하는지 체감했으며, 데이터베이스와 애플리케이션 간의 적절한 역할 분담이 시스템 성능에 결정적인 영향을 미친다는 것을 배웠습니다.

주요 변경사항

- 단편적으로 나열된 문제 원인(12번 호출, 함께 쿼리 등)을 서론의 문제 정의 부분으로 통합하여 문맥 구축
- List loop 변경과 DTO 도입을 '애플리케이션 계층의 역할 강화'라는 논리적 판단 근거와 연결
- 표로 제시된 성능 개선 수치(시간 ms, 쿼리 개수 비율 %)를 성과 단락에 서술형으로 명시하여 가독성 강화
- 이미지 컨텍스트(Service Layer 1/2)를 활용하여 테스트 코드가 수정 없이 통과할 수 있었던 아키텍처적 근거 추가
- 추상적인 깨달음('용기를 내지 않으면')을 아키텍처 설계와 역할 분담의 관점(직무 기여 방향)으로 구체화

전후 점수 82.0 → 93.2 (+11.2) 양호 ★★ → 우수 ★★★

A.과정/판단력 75→90(+15) | B.역할/기여도 85→95(+10) | C.성과/인사이트 90→95(+5) | D.작성품질 8→10(+2) | E.직무연관성 85→90(+5)

[성과]

원문 (284자)

프로젝트 구조에 대한 고민과 구현

- 하나의 컨트롤러 안에 여러 서비스 메서드를 호출하는 경우가 발생하였는데, 그래도 괜찮을까? 에 대한 의문으로 시작하여 프로젝트

- 구조에 대해 고민하게 됨 (저장버튼이 1개라서 addIncome 과 updateIncome 메서드가 함께 실행되어야 했음)

- 서비스 Layer를 나누어 관리하기로 결정하고 두 메서드를 묶는 서비스 클래스를 하나 더 둠

- 아래 그림처럼 구조를 가져감으로 Service에서의 순환참조나 순환구조를 애초에 방지하도록 설계함

개선안 (783자)

****프로젝트 구조 최적화를 위한 서비스 계층(Layer) 분리 및 순환 참조 방지 설계****

단일 저장 버튼 클릭 시 addIncome 과 updateIncome 등 여러 서비스 메서드가 함께 실행되어야 하는 요구사항이 있었습니다. 초기에는 하나의 컨트롤러 내에서 여러 서비스 메서드를 연속적으로 호출하는 방식을 직관적으로 떠올렸으나, 이러한 구조가 적절한지에 대한 강한 의문이 들었습니다. 컨트롤러가 비즈니스 로직에 과도하게 관여하게 되어 결합도가 높아지고 향후 유지보수성이 저하될 우려가 있었기 때문입니다.

이러한 구조적 문제를 해결하기 위해, 서비스 계층(Service Layer)을 다계층으로 분리하여 관리하는 아키텍처를 도입했습니다. 구체적으로 addIncome 과 updateIncome 등 하위 서비스 로직을 통합하여 조율하는 상위 서비스 클래스(Service Layer 1)를 새롭게 정의했습니다. 데이터베이스 리포지토리에 직접 접근하는 책임을 하위 서비스 클래스(Service Layer 2)에만 부여함으로써, 계층 간의 역할과 접근 규칙을 명확히 분리했습니다.

이러한 구조 개편을 통해 컨트롤러의 부담을 줄이고 두 메서드를 하나의 응집된 서비스 흐름으로 묶어낼 수 있었습니다. 또한, 명확한 단방향 의존성 규칙을 수립하여 서비스 간의 순환 참조나 복잡한 순환 구조 발생을 원천적으로 방지하는 안정적인 시스템을 설계했습니다. 이 과정을 통해 단순한 기능 구현을 넘어, 시스템의 안정성과 확장성을 담보하는 아키텍처 설계가 비즈니스 요구사항 해결에 필수적임을 체감했습니다.

주요 변경사항

- 단일 버튼 클릭으로 인한 다중 서비스 호출 상황을 문제 정의로 명확히 제시
- 아키텍처 다이어그램 컨텍스트를 활용해 서비스 계층(Layer 1, Layer 2) 분리 과정을 구체화
- 순환 참조 방지 및 계층별 책임 분리라는 아키텍처 설계의 목적과 성과 강조

- 단순 사실 나열 형태의 개조식을 '문제-판단-실행-인사이트' 구조의 서술형으로 재배치

전후 점수 62.8 → 82.8 (+20.0) 보통 ★ → 양호 ★★

A.과정/판단력 65→85(+20) | B.역할/기여도 60→80(+20) | C.성과/인사이트 50→70(+20) | D.작성품질 8→10(+2) | E.직무연관성 70→90(+20)

[개발_0]

원문 (3110자)

서버 배포와 도커, CI/ CD 경험

- · CI/CD를 사용하기 위해 Jenkins를 프로젝트에 적용해야 했기 때문에 스스로 학습하고 프로젝트에 적용함

- · Github에 PR이 Merge되면 webhooks 를 활용하여 Jenkins의 파이프라인 가동

- Git Clone - TestCode Check Jar Build Docker Image Build Docker Image Push & Pull Docker Image Run 순서로 CI/ CD 진

- 행 가계부 포트폴리오 ○○○

9

GitHub 오전 4:40

Pull request opened by jiyongYoon

#213 REFACTOR/Fetch를 Lazy로 하여 쿼리 개선

변경사항

AS-IS

· @manytoone(fetch = FetchType.LAZY) 설정하여 쿼리 개선

TO-BE

테스트

테스트 코드

API 테스트

Labels

Refactor

RE-ZERO-In-And-Out/In-And-Out 오늘 오전 4:40

Pull request merged by jiyongYoon

#213 REFACTOR/Fetch를 Lazy로 하여 쿼리 개선

○ RE-ZERO-In-And-Out/In-And-Out 오늘 오전 4:40

jenkins 앱 오전 4:44

SUCCESSFUL: Deploy http://ec2-3-34-206-181.ap-northeast-2.compute.amazonaws.com:3000/

깃헙에 머지되면 CI/CD 후 Notice가 오는 모습 직접 작성한 Jenkins Pipeline 스크립트

pipeline {

```

environment {

    repository = "a/backend"
    DOCKERHUB_CREDENTIALS = credentials( 'dockerhub-jenkins' )
    dockerImage = , ,

}

agent any stages {

    stage( github clone 1 ) {

        steps {

            git branch: 'main credentialsId: 'github-jenkins',
            url: 'https://github.com/RE-ZERO-In-And-Out/In-And-Out.git echo 'git clone finish'

        }

    }

    stage( build jar' ){

        steps{

            dir( server 1 ) {

                # echo 'build start ,
                sh ' , ,
                sudo cp /home/ubuntu/properties/app lication-dbsecret.properties /var/lib/jenkins/workspace/inandout-jenkin
                sudo cp /home/ubuntu/properties/app lication-secret.properties /var/lib/jenkins/workspace/inandout-jenkins/
                sudo chmod +x gradlew
                /gradlew clean build
                ■ ■ ,

            # }

            # }
            post{

                # success {

                    echo ***** jar build success*****

                # }
                failure {

                    error ***** jar build failed & pipeline stops* *****

                # }
            }
        }
    }
}

```



```

# }

# }

# stage( docker image build' ){

# steps{

dir( /var/lib/jenkins/workspace/Jenkins-pipeline/server'){

echo image build start ·
sh ' , ,
sudo chmod 666 /var/run/ docker. sock
sudo cp /home/ubuntu/dockerfile/Dockerfile /var/lib/jenkins/workspace/ Jenkins-pipeline/server/Dockerfile
sudo docker build -t inandout
docker tag inandout: latest a/backend: inandout
■ ■ ,

# }

}
post{

# success {

echo *****docker image build success *****

가계부 포트폴리오 ○ ○ ○

10

# }
failure {

# error *****docker image build failed & pipeline stops *****

# }

# }

# }

# stage( 'Login_jenkins server'){

# steps{

}

sh 'echo $DOCKERHUB_CREDENTIALS_PSW | docker login -u $DOCKERHUB_CREDENTIALS_USR ·
-password-stdin'

```

```

# }

# stage( Deploy our image · ) {

# steps {

# script {

# sh ' docker push $repository :inandout ,

# }

# }

# }

# stage( 'Cleaning up ' ) {

# steps {

sh "docker rmi $repository :inandout"

# }

}

# stage( 'Prepare'){

# steps{

sshagent([ 'c3e3d23e-e1f8-4102-90c9-a79382e83bf8 · ]) {

sh "ssh -0 StrictHostKeyChecking=no ubuntu@3.34.206.181 · sudo chmod 666 /var/run/docker.sock"
sh "ssh -0 StrictHostKeyChecking=no ubuntu@3.34.206.181 'docker images · "

}

}

}

stage( Login_boot server'){

steps{

sshagent([ 'c3e3d23e-e1f8-4102-90c9-a79382e83bf8']) {

sh ' echo $DOCKERHUB_CREDENTIALS_PSW | ssh -0 StrictHostKeyChecking=no ubuntu@3.34.206.181 docker
login -u

}

```

```
}
```

```
}
```

```
stage( Docker Run ) {
```

```
steps {
```

개선안 (991자)

[프로젝트경험 — In-And-Out 가계부 서비스: CI/CD 자동화 파이프라인 구축 및 쿼리 최적화]

수동 배포로 인한 비효율적인 리소스 낭비를 막고 개발 생산성을 높이기 위해 자동화된 배포 파이프라인을 구축했습니다. CI/CD 도구로 Jenkins를 채택하여 주도적으로 학습하고 프로젝트에 적용했습니다.

먼저, GitHub 레포지토리에 PR이 Merge되면 Webhook을 통해 Jenkins 파이프라인이 자동으로 가동되도록 연결했습니다. 파이프라인은 'Git Clone → TestCode Check → Jar Build → Docker Image Build → Docker Image Push & Pull → Docker Image Run'의 체계적인 흐름으로 직접 스크립트를 작성해 설계했습니다. 이 과정에서 민감한 정보가 담긴 application-dbsecret.properties 등의 설정 파일은 보안을 위해 레포지토리와 분리하고, 빌드 단계에서 동적으로 복사해 사용하는 방식을 택하여 안전성을 높였습니다. 이후 빌드된 Jar 파일을 기반으로 Docker Image를 생성하고 Docker Hub에 Push한 뒤, SSH 접속을 통해 운영 서버에서 최신 이미지를 Pull하여 실행하도록 구현했습니다. 파이프라인 실행 완료 후에는 성공 여부가 실시간으로 Notice(알림) 되도록 연동하여 운영 모니터링 환경을 개선했습니다.

인프라 구축뿐만 아니라 애플리케이션의 쿼리 최적화도 주도했습니다. PR #213 리팩토링을 통해 기존 엔티티 매핑에서 발생하던 불필요한 연관 데이터 조회를 방지하고자, @ManyToOne 어노테이션의 옵션을 FetchType.LAZY 로 변경하여 쿼리 효율을 개선했습니다.

이 경험을 통해 Jenkins 스크립트 작성부터 Docker 기반의 컨테이너 배포까지 전체적인 인프라 구축 사이클을 경험했습니다. 단순한 기능 개발을 넘어 파이프라인 자동화와 쿼리 튜닝이 서비스 운영 효율성에 직결된다는 인사이트를 얻었습니다.

주요 변경사항

- 단순 코드 덤프 및 단답형 나열을 논리적인 서술형 단락으로 재구성
- Jenkins 스크립트 내부에서 확인되는 properties 파일 복사 과정을 보안 목적의 판단 근거로 명시
- GitHub PR #213에 언급된 'FetchType.LAZY' 설정 변경을 쿼리 최적화 성과로 명확히 연결
- 빌드/배포 알림(SUCCESSFUL 메시지) 연동 내용을 모니터링 환경 개선 성과로 구체화
- 목적-과정-성과-인사이트의 포트폴리오 구조(STAR 기법)에 맞게 내용 재배치

전후 점수 63.8 → 79.5 (+15.8) 보통 ★ → 양호 ★★

A.과정/판단력 60→75(+15) | B.역할/기여도 75→85(+10) | C.성과/인사이트 50→70(+20) | D.작성품질 7→10(+3) | E.직무연관성 70→80(+10)

[개발_1]

원문 (1287자)

```
echo 'Pull Docker Image & Docker Image Run '
sshagent([ 'c3e3d23e-e1f8-4102-90c9-a79382e83bf8' ]) {
```

```

sh "ssh -O StrictHostKeyChecking=no ubuntu@3.34.206.181 · docker pull a/backend : inandout ..."
sh "ssh -O StrictHostKeyChecking=no ubuntu@3.34.206.181 · docker ps -q --filter name=inandout | grep -q &
sh "ssh -O StrictHostKeyChecking=no ubuntu@3.34.206.181 'docker run -d = -name inandout -p 8080:8080
a5/bac

}

}
post{

success {

echo *****docker image run success*****

}
failure {

}

# error *****docker image run failed & pipeline stops*****

}

}

}

post {

success {

slackSend (

channel : ' #inandout-pr' ,
color: , #00FF00' ,
message: "SUCCESSFUL: Deploy http://ec2-3-34-206-181.ap-northeast-2.compute.amazonaws.com:3000/" )

}
failure {

slackSend (

channel : · #inandout-pr ,
color: ' #FF0000 ·
`
message: "FAILED : Job '${env.JOB_NAME}' [${env.BUILD_NUMBER}] · (${env.BUILD_URL})"

}

}
}

```

}

Github 브랜치 컨벤션 및 PR 활성화, Mock Test와 API 테스트 진행.

· 프로젝트 현재 총 200개의 pr이 Closed 됨.

가계부 포트폴리오 ○ ○ ○

11

- pr 중 1/5 이상은 코드리뷰를 통해 코드 구조가 변경되거나 오류가 잡히고 효율이 개선되는 경험을 함.
- 코드에서 좋은 부분을 습득하여 내 코드에 적용하는 경우도 있었음.
- 브랜치 컨벤션과 라벨, pr-template을 활용하였고, AS-IS / TO-BE / TEST 세 부분으로 구성하여 관리하였음.
- 이를 통해 테스트 코드가 깨지거나 부트가 빌드되지 않는 등의 비효율을 최소화함.

▼ 협업 툴을 활용하여 업무 효율 극대화

개선안 (840자)

[프로젝트 경험 — In-and-Out: 가계부 서비스 협업 및 CI/CD 파이프라인 구축]

팀 프로젝트인 가계부 서비스 'In-and-Out'을 개발하며, 잦은 코드 병합으로 인한 빌드 실패와 수동 배포의 비효율을 해결하고자 배포 자동화 및 체계적인 협업 프로세스를 구축했습니다.

먼저, Jenkins와 Docker를 활용하여 백엔드 컨테이너 이미지를 풀(Pull)하고 실행하는 자동 배포 파이프라인을 구성했습니다. 이와 함께 파이프라인의 성공 및 실패 결과를 팀의 Slack 채널(#inandout-pr)로 실시간 전송하도록 설정하여, 배포 상태에 대한 팀 내 가시성을 확보하고 이슈 발생 시 신속한 대응이 가능하도록 만들었습니다.

코드 병합 단계에서의 안정성을 확보하기 위해 GitHub 브랜치 컨벤션과 라벨링 시스템을 도입했습니다. 특히 PR 템플릿을 'AS-IS / TO-BE / TEST' 세 가지 항목으로 구조화하여 변경 사항과 목적을 명확히 공유하도록 규정했습니다. 병합 전 Mock 테스트와 API 테스트를 반드시 진행하도록 하여, 테스트 코드가 깨지거나 부트가 빌드되지 않는 등의 비효율적인 상황을 최소화했습니다.

이러한 체계적인 협업 환경을 바탕으로 프로젝트 기간 동안 총 200개의 PR을 관리 및 Closed 처리했습니다. 전체 PR 중 1/5 이상은 적극적인 코드 리뷰를 거치며 코드 구조가 개선되고 잠재적인 오류를 사전에 차단하는 성과를 얻었습니다. 동료의 우수한 코드를 리뷰하며 이를 제 코드에 적용하는 등 개인의 역량 성장도 이뤘습니다. 파이프라인 자동화와 명확한 리뷰 문화가 팀의 업무 효율을 극대화하고 서비스 품질을 유지하는 핵심 기반임을 깨달은 경험이었습니

주요 변경사항

- Jenkins 스크립트와 Slack 연동 코드를 '자동 배포 및 실시간 상태 공유 파이프라인 구축'이라는 구체적 성과로 문장화함
- '총 200개의 PR', '1/5 이상' 등의 원문 수치를 유지하며 이를 코드 리뷰와 품질 개선의 결과로 연결함
- 단순 나열된 PR 템플릿(AS-IS/TO-BE/TEST)과 테스트 경험을 '빌드 실패와 비효율 최소화'를 위한 판단 근거 및 실행 과정으로 재조직함
- '좋은 부분을 습득함', '극대화' 등의 추상적 표현을 협업 과정에서의 성장과 인사이트 도출로 발전시킴

전후 점수 61.2 → 82.2 (+21.0) 보통 ★ → 양호 ★★

A.과정/판단력 55→75(+20) | B.역할/기여도 60→80(+20) | C.성과/인사이트 65→85(+20) | D.작성품질 7→10(+3) | E.직무연관성 70→90(+20)

[성과_0]

원문 (1248자)

▼ 노션 회의록을 사용하여 진행사항을 체계적으로 정리하고 관리함 회의록

- Chart Type: line

| | Westy Recreq | Sprint Fuel 1 (2 m) | Daily Score 2 (10.19) | Daily Score 4 (10.20) | Daily Score 3 (10.21) |
Westy Rock 2 (2.0wk) | Sprint Plan 2 (3.week) | Daily Score 6 (10.25) | Daily Score 7 (10.25) | Daily Score 8 (10.25)
| Daily Score 9 (10.27) | Daily Score 10 (18.25) | Weekly Score 9 (3. Weekly Score 10) | Sprint 7 (3.4) | Fresh Pass 2
(4.week) | Daily Score 11 (10.31) | Daily Score 12 (11.01) | Daily Score 13 (11.03) | Daily Score 15 - (11.04) |
Weekly Review 3 (4.week) | Spirit Routes 4 (3.44) | Daily Score 15 - (11.07) | Daily Score 17 - (11.08) | Daily Score
13 - (11.09) | Daily Score 20 (1.1) | Weekly Review 3 (0.44) | Weekly Review 5 (5.6) | Sports Routes 4 (5.06) |
Sports Routes 10 (1.1) | Sport Routes 10 (1.1) |

| --- |
| --- | --- | --- |

| item_01 | 10% | 10% | 10% | 10% | 10% | 10% | 10% | 10% | 10% | 10% | 10% | 10% | 10% | 10% | 10% | 10% |
10% | 10% | 10% | 10% | 10% | 10% | 10% | 10% | 10% | 10% | 10% | 10% | 10% |

▼ 노션 페이지를 활용하여 API 수정사항 누락방지 가계부 포트폴리오 ○○○

12

API 테스트 수정요청사항

개선안 (676자)

[프로젝트경험 — 가계부 포트폴리오 프로젝트 협업 프로세스 구축]

가계부 애플리케이션 개발 과정에서 팀원 간 작업 진행 상황 파악이 어렵고, API 테스트 후 수정 요청 사항이 누락되는 문제가 발생했습니다. 이를 해결하고 개발 효율을 높이고자 노션(Notion)을 활용하여 체계적인 프로젝트 관리 환경을 구축했습니다.

우선, 작업 진행 상황을 명확히 가시화하기 위해 노션 회의록을 도입했습니다. 스프린트 계획, 데일리 스크럼, 주간 리뷰를 꾸준히 기록하여 팀의 목표와 일일 작업 내역을 추적했습니다. 더불어 태스크 카드를 칸반보드(0%, 25%, 50%, 75%, PR, 100% / Merge)와 타임라인 뷰로 시각화하여 단계별 작업 흐름과 일정을 체계적으로 관리했습니다.

또한, API 수정 과정에서의 소통 오류를 줄이고자 전용 노션 페이지를 신설했습니다. 'API 테스트 수정 요청 사항'을 추적하기 위해 요청 내용, 요청자, 담당자, 진행 상황 및 해결일을 명확히 기록하는 테이블을 구성하여 수정 사항 누락을 사전에 방지했습니다.

결과적으로 노션을 통한 체계적인 문서화와 태스크 관리를 통해 API 수정 누락 없이 안정적으로 개발 일정을 소화할 수 있었습니다. 이 경험을 통해 명확한 작업 추적과 기록 시스템이 프로젝트 품질 향상과 원활한 협업에 미치는 긍정적인 영향을 깊이 이해하게 되었습니다.

주요 변경사항

- 결과만 나열된 원문을 '문제 정의 → 실행 과정 → 성과 및 인사이트' 구조로 재배치함
- 진행 사항 파악 및 API 수정 누락 방지라는 명확한 노션 도입 목적(판단 근거)을 추가함
- 이미지 컨텍스트에 명시된 스크럼(데일리 스크럼, 주간 리뷰 등) 및 칸반보드(0~100%) 활용 내용을 구체적인 실행 방안으로 서술함
- 태스크 관리 항목(요청자, 담당자, 진행 상황 등)을 명시하여 문제 해결 과정을 구체화함
- 체계적인 문서화와 작업 추적이 프로젝트 품질과 협업에 미치는 영향을 인사이트로 연결함

전후 점수 31.8 → 72.8 (+41.0) 미흡 → 양호 ★★

A.과정/판단력 30→75(+45) | B.역할/기여도 25→70(+45) | C.성과/인사이트 20→60(+40) | D.작성품질 8→10(+2) | E.직무연관성 30→70(+40)

[성과_1]

원문 (1833자)

■ 표 필터 정렬 a 새로 만들기 V	■ 표 필터 정렬 a 새로 만들기 V	■ 표 필터 정렬 a 새로 만들기 V	■ 표 필터 정렬 a 새로 만들기 V	■ 표 필터 정렬 a 새로 만들기 V	■ 표 필터 정렬 a 새로 만들기 V
제목 없음	제목 없음	제목 없음	제목 없음	제목 없음	제목 없음
Aa 항목	22 요청자	요청일	진행상황	22 담당자	해결일
h 회원 정보 불러올 때 오류 h 비밀번호 변경 CORS 오류 (수입 지출 등록에도 동일한 오류 발생)	윤 윤지용	2022년 11월 10일	Done	김찬주 윤 윤지용	2022년 11월 14일
명 Calendar에서 지출금액 조회오류	[c]윤[c] 윤지용 윤지용	2022년 11월 10일	Done	김찬주	2022년 11월 11일 2022년 11월 14일
명 다이어리 get 요청 보냈을 때 지출 데이터 하나만 받아짐	9 LTH	2022년 11월 11일	Done		
		2022년 11월 12일	Done	윤지용	2022년 11월 12일
명 수입/지출 테이블 수정 오류	윤 윤지용	2022년 11월 12일	Done	김찬주	2022년 11월 12일
연간 보고서 카테고리 수정	윤지용	2022년 11월 12일	● Done	김찬주	2022년 11월 12일
h INCOME-DELETE-삭제안됨	[c]윤[c] 윤지용	2022년 11월 14일	● Done	김찬주	2022년 11월 14일
h Expense delete 안됨	근형 G.H. Oh	2022년 11월 14일	Done	김찬주	2022년 11월 14일
h Excel-Monthly-Export h Table-수입지출 금액 입력 시 0값, 날짜 연동	[c]윤[c] 윤지용 [c]윤[c] 윤지용 G.H. Oh	2022년 11월 14일 2022년 11월 14일	Done	김찬주 김찬주	2022년 11월 14일 2022년 11월 14일
명 Report 보고서 파이형 그래프 수정	[c]윤[c] 윤지용	2022년 11월 14일	● Done ● Done	김찬주	2022년 11월 14일
명 EXCEL-GET-YEAR	[c]유[c] 윤지용	2022년 11월 14일	Done	김찬주	2022년 11월 14일
명 Member-setting-프로필 변경시 사진이 없어짐	윤 윤지용	2022년 11월 15일	Done	김찬주	2022년 11월 17일
h Table-수입/지출테이블에 0을 지우게 되면 null이 전달되는 상황	[c]윤[c] 윤지용	2022년 11월 15일	Done	김찬주	2022년 11월 15일
Table-매달 1일이 날짜 입력이 안 됨	윤지용	2022년 11월 15일	Done	김찬주	2022년 11월 15일
Calendar-화면이 안뜨는 기간존재 □2	윤 윤지용	2022년 11월 15일	Done	LTH	2022년 11월 15일
비밀번호 찾기 버튼			Done	김찬주	2022년 11월 16일
+ 새로 만들기	+ 새로 만들기	+ 새로 만들기	+ 새로 만들기	+ 새로 만들기	+ 새로 만들기

▼ 칸반 보드를 활용하여 업무일정 관리 가계부 포트폴리오 ○ ○ ○

13

프로젝트 일정관리

개선안 (993자)

[프로젝트 경험 — 가계부 플랫폼 개발 및 일정 관리]

React와 Spring Boot 기반의 가계부 및 다이어리 플랫폼을 개발하며, 백엔드 아키텍처 설계와 칸반 보드 기반의 애자일 프로젝트 관리에 집중했습니다.

유지보수성과 역할 분리를 위해 Controller, Service(Layer 1, Layer 2), Query Repository로 구성된 다계층 아키텍처를 도입했습니다. 연간 보고서 데이터를 처리할 때 Report Service(Layer 1)가 Income/Expense Service(Layer 2)를 통해 각각의 Repository에서 데이터를 조회하도록 계층 간 접근 규칙을 엄격히 적용했습니다.

데이터베이스는 MySQL(AWS RDS)과 Redis를 활용하였으며, AWS EC2 및 Docker 기반의 배포 파이프라인(GitHub, Jenkins)을 구축했습니다.

팀 프로젝트의 협업 효율을 높이기 위해 태스크 진행 상황을 0%, 25%, 50%, 75%, PR, 100% / merge 단계로 칸반 보드에 시각화하고, 2022년 10월부터 11월까지 데일리 스크럼과 주간 리뷰를 진행했습니다. 이슈 트래커를 통해 캘린더 지출 조회 오류, 매달 1일 날짜 입력 불가 현상, 수입/지출 금액 입력 시 0을 지우면 null이 전달되는 예외 현상 등 데이터 무결성 관련 버그를 체계적으로 추적하여 완료(Done) 처리했습니다.

또한 프론트엔드 연동 중 발생한 비밀번호 변경 PATCH 요청의 CORS 차단 오류를 분석하여 해결했습니다. 더불어 ColConfig 객체를 활용해 엑셀 월별(Excel-Monthly-Export) 및 연간(EXCEL-GET-YEAR) 데이터 다운로드 기능을 Java로 직접 구현하며, 예외 발생 시 안전하게 예외 처리가 되도록 구성했습니다. 이러한 체계적인 이슈 추적과 계층형 백엔드 설계를 바탕으로 기능 개발을 완료했으며, 지속적인 모니터링과 프로세스 관리가 애플리케이션의 안정성을 높인다는 인사이트를 얻었습니다.

주요 변경사항

- 단순 표 형식으로 나열된 이슈 트래킹 로그를 '프로젝트 목적 - 역할 - 실행 - 성과' 구조의 서술형 문장으로 변환
- 이미지 컨텍스트에 포함된 다계층 아키텍처(Layer 1, Layer 2), 기술 스택(Spring Boot, AWS EC2, Docker 등) 정보를 통합하여 기술적 깊이 보완
- 비밀번호 CORS 오류 해결, 수입/지출 입력 시 null 전달 오류 해결, 엑셀 다운로드 기능 구현 등 구체적인 트러블슈팅 경험 부각
- 칸반 보드 진행 상황 수치(0%, 25%, 50%, 75%, 100%)와 개발 일정(2022년 10월~11월)을 명시하여 프로젝트 관리 역량 강조

전후 점수 28.5 → 76.8 (+48.2) 미흡 → 양호 ★★

A.과정/판단력 20→75(+55) | B.역할/기여도 30→80(+50) | C.성과/인사이트 10→60(+50) | D.작성품질 8→10(+2) | E.직무연관성 40→85(+45)

[이슈]

원문 (2718자)

▼ 협업 마인드

-- 어떤 이슈가 생기거나, 도전해 보고 싶은 부분이 생겼을 때 항상 'YES'인 마인드로 임하였습니다.

-- 나중에 글을 읽은 팀원들이 요청자에게 질문했을 때 'ㅇㅇ님이 해결해주셨습니다'라는 피드백을 많이 받았다고 합니다.

-- API 수정 요청 시 문제를 정확하게 전달하기 위해 Network HTTP 요청을 확인하고, 해당 이슈로 추론되는 부분의 내용을 검색해

- 보고 공유하여 함께 해결함

-- 이를 통해 프론트엔드 개발자의 관심사와 지식 및 업무 범위에 대한 내용을 알게 됨 가계부 포트폴리오 ○○○

14

ICUL ULL // 비ㅅㅌ □ □ □ ○

| 서환타 0 OTWelcomeRA.woff2 | 국민의 □ favicon.ico □ manifestjson □ logo192.png □ info ■ password | User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/53 Request Headers O dataimage/jpeg.bas... logo.b79d1776d25fe8e5e17f ▲ Provisional headers are shown Learn more Accept: application/json, text/plain, */* Content-Type: application/json Referer: http://3.34. 206,181: 3000/ 6 (KHTML, like Gecko) Chrome/107 .0.0.0 Safari/537.36 □ password □ password □ password □ password ■ password password password requests

1.5 |
| --- | --- | --- |
| | 19 MB transferred 1.8 MB resources ... Console Issues top Filter Default levels 5 Issues: 1 3 hidden ▶ PATCH http://3.34.206, 181:8080/aoi/member/password net::ERR_FAILED ahr. is:244 foLdPw: 'bLa123!@', PW2 'black1231@# passwordConfirm: 'black123!@#') PasswordChange. is:78 Access to XMLHttpRequest at http://3.34.206,181:8080/aol/member/pessword' from origin "hit cassword change:1 p://3.34.206,181:3000 has been blocked by CORS policy: Response to preflight request doesn't pass access control check: Redirect is not allowed for 구 preflight request. PasswordChange. js:70 To {message: "Network Error name: 'AxlosError code: 'ERR_NETWORK', config: (m), request: XPILHttpReques 파 ▶ PATCH http://3.34.206,181:8080/aoi/member/password net::ERR_FAILED xhc. is:244 | 19 MB transferred 1.8 MB resources ... Console Issues top Filter Default levels 5 Issues: 1 3 hidden ▶ PATCH http://3.34.206, 181:8080/aoi/member/password net::ERR_FAILED ahr. is:244 foLdPw: 'bLa123!@', PW2 'black1231@# passwordConfirm: 'black123!@#') PasswordChange. is:78 Access to XMLHttpRequest at http://3.34.206,181:8080/aol/member/pessword' from origin "hit cassword change:1 p://3.34.206,181:3000 has been blocked by CORS policy: Response to preflight request doesn't pass access control check: Redirect is not allowed for 구 preflight request. PasswordChange. js:70 To {message: "Network Error name: 'AxlosError code: 'ERR_NETWORK', config: (m), request: XPILHttpReques 파 ▶ PATCH http://3.34.206,181:8080/aoi/member/password net::ERR_FAILED xhc. is:244 |

예상원인(선택)

- 요 블로그 보면 spring security에서 해당 option request를 redirect 시켜서 그런것 같다고 하네
- 요 (링크)
- 위 블로그 보다는 csrf 설정이 enabled 되어 있어서 안되서 그렇다는 것이 더 신빙성이 있는 것
- 같습니다 (링크)

API 수정 요청 내용 가계부 포트폴리오 ○ ○ ○

15

□ 커버 추가 - 댓글 추가

API 테스트 수정요청사항 개발 후 API 테스트 시 모든 테스트들을 성실하게 수행하여 가장 많은 수정요청사항을 게시함

개선안 (757자)

[프로젝트경험 — 가계부 플랫폼 API 테스트 및 이슈 추적 최적화]

가계부 포트폴리오 프로젝트에서 API 연동 테스트를 수행하며 프론트엔드와 백엔드 간의 이슈 해결 프로세스를 개선했습니다. 기존에는 에러 발생 시 단순 현상만 전달하여 디버깅에 오랜 시간이 소요되는 문제가 있었습니다. 저는 비밀번호 변경 PATCH 요청 시 CORS 에러가 발생하는 것을 발견했을 때, 브라우저 개발자 도구의 Network 탭과 콘솔 로그를 직접 분석했습니다. Preflight 요청 단계에서 접근 제어 확인을 통과하지 못해 리다이렉트 에러가 발생함을 직시했습니다.

이를 바탕으로 Spring Security의 OPTIONS 요청 리다이렉트 처리 방식이나 CSRF 설정 옵션이 원인일 수 있다는 가설을 세웠습니다. 단순 수정 요청에 그치지 않고 예상되는 원인과 관련 기술 블로그 레퍼런스를 덧붙여 담당자에게 공유했습니다. HTTP 요청 데이터를 기반으로 명확하게 문제를 전달한 결과, 팀원의 디버깅 시간을 단축시키고 신속하게 문제를 해결하는 데 기여했습니다.

이러한 방식으로 프로젝트 기간 동안 모든 API 테스트를 성실하게 수행하여 가장 많은 수정 요청 사항을 체계적으로 게시하고 관리했습니다. 적극적으로 원인을 분석하고 공유하는 태도를 통해 팀원들로부터 문제 해결에 큰 도움을 주었다는 피드백을 받았습니다. 이 경험을 통해 프론트엔드 개발자의 관심사와 업무 범위를 깊이

이해하게 되었으며, 근거 기반의 명확한 소통이 협업의 생산성을 극대화한다는 것을 배웠습니다.

주요 변경사항

- 추상적인 자기 평가('YES 마인드') 삭제 및 구체적인 문제 해결 행동으로 대체
- 이미지 컨텍스트의 CORS 에러, Spring Security, CSRF 원인 분석 내용을 핵심 성과로 반영
- 단순 '요청했다'를 넘어 'Network HTTP 요청을 확인하고 예상 원인을 공유했다'는 과정의 가치를 부각
- 가장 많은 수정 요청 사항을 게시한 경험을 꼼꼼한 API 테스트 및 주도적 태도의 결과로 연결
- '프론트엔드의 관심사를 알게 됨'을 협업 프로세스 최적화를 통한 인사이트로 발전시킴

전후 점수 55.8 → 85.8 (+30.0) 보통 ★ → 우수 ★★★

A.과정/판단력 55→85(+30) | B.역할/기여도 60→90(+30) | C.성과/인사이트 40→75(+35) | D.작성품질 7→10(+3) | E.직무연관성 65→85(+20)

[성과]

원문 (590자)

프로젝트에서 개선하거나 추가로 하고 싶은 부분

- 관리자 페이지를 추가로 구현하고 싶습니다. 현재는 카테고리 추가를 하는 부분이 DB에서만 가능하기 때문에 DB접근 없이 온전
- 히 서비스가 작동된다고 하기에는 부족하다고 생각합니다. 관리자 페이지에는 카테고리 추가, 삭제 부분을 메인으로 하고 유저관
- 리(활성화, 비활성화, 아이디와 비밀번호 오류 등의 내용 처리가 가능하도록) 기능을 추가해보고 싶습니다.
- 맨 처음 기획했던 방향인 [가게부 + SNS] 서비스를 추가해보고 싶습니다. 사실 프로젝트 초반에는 내가 작성한 가게부를 SNS 형
- 식으로 공유하고 서로 피드백을 주고받는 서비스를 개발해보고 싶었지만, 저희의 역량과 기간, 그리고 완성도 등의 모든 것을 고려
- 하여 범위를 조정하게 되었습니다. 결과적으로는 정말 잘한 선택이라고 생각합니다. 덕분에 더 긴시간 API 테스트를 하고 서버 배
- 포도 더 구체적으로 할 수 있었고, 가게부 기능을 조금 더 완성도 있게 가져갈 수 있었습니다. 기간에 따른 목표치를 잘 설정하는
- 것도 꼭 필요한 역량이라고 생각합니다.

읽어주셔서 감사합니다 가게부 포트폴리오 ○○○

16

개선안 (698자)

[향후 개선 과제 및 프로젝트 회고]

첫째, 관리자 페이지 구축을 통한 서비스 운영의 완전성 확보를 목표하고 있습니다. 현재는 데이터베이스(DB)에 직접 접근해야만 카테고리 추가가 가능하여 서비스의 독립적인 운영에 한계가 존재합니다. 이를 개선하기 위해 카테고리 추가 및 삭제 기능을 메인으로 하는 관리자 페이지를 구현하고자 합니다. 더불어 유저 관리(계정 활성화/비활성화, 아이디 및 비밀번호 오류 처리 등) 기능을 도입하여 DB 의존도를 낮추고 안정적인 서비스 운영 환경을 마련하고 싶습니다.

둘째, 프로젝트 초기 기획 단계에서 구상했던 '가게부 기반 SNS 서비스'로의 확장을 고려하고 있습니다. 기획 초기에는 사용자가 작성한 가게부를 SNS 형태로 공유하고 상호 피드백을 주고받는 기능을 목표로 했습니다.

하지만 한정된 개발 기간과 팀의 역량, 그리고 핵심 기능의 완성도를 종합적으로 판단하여 개발 범위를 가계부 본연의 기능으로 축소하는 결정을 내렸습니다.

이러한 선택은 결과적으로 핵심 기능의 완성도를 높이는 데 기여했습니다. 축소된 범위 덕분에 API 테스트에 더 긴 시간을 투자하고, 서버 배포 과정을 더 구체화하며 서비스 안정성을 검증할 수 있었습니다. 이 과정을 통해 프로젝트의 완성도를 높이기 위해서는 정해진 기간과 가용 자원을 정확히 파악하고, 그에 맞춰 현실적인 목표치를 설정하는 판단력이 필수적인 역량임을 깨달았습니다.

주요 변경사항

- 구어체 표현을 전문적인 회고 및 향후 계획 형태의 문장으로 수정
- 관리자 페이지 도입과 SNS 기능 확장이라는 두 가지 주제로 문단을 분리하여 가독성 향상
- 개발 범위 조정에 대한 과정을 판단 근거, 실행 결과, 깨달음(인사이트)의 논리적 흐름으로 재배치
- 불필요한 단순 맺음말을 생략하고 직무 역량 관점에서의 회고에 집중

전후 점수 57.8 → 77.8 (+20.0) 보통 ★ → 양호 ★★

A.과정/판단력 65→85(+20) | B.역할/기여도 50→70(+20) | C.성과/인사이트 40→65(+25) | D.작성품질 9→10(+1) | E.직무연관성 55→75(+20)
